
⚡ Amazon DynamoDB Accelerator (DAX)

🧠 What is DAX?

♦ **Amazon DAX (DynamoDB Accelerator)** is a **fully managed, in-memory caching service** for DynamoDB.

- ⚡ Boosts performance from **milliseconds** ➡ **microseconds**
- 🧠 Acts as a **write-through** or **read-through** cache
- 📦 Built specifically for **DynamoDB queries**

📌 Think of it like **Redis**, but purpose-built to work with DynamoDB — with **no code changes** if you use AWS SDKs!

🔧 Key Features of DAX

Feature	Description
⚡ Microsecond latency	10x performance boost for reads
🧠 Write-through cache	Keeps DynamoDB and cache in sync
☁ Fully managed	No servers to patch or maintain
🔒 Secure	Integrated with VPC, IAM, KMS
🔄 Compatible	Works with existing DynamoDB SDK calls



Scalable

Add nodes or clusters easily



Read-heavy
optimization

Best for read-intensive workloads



How DAX Works

App → AWS SDK → DAX Cluster → DynamoDB

- First read? 📖 → DAX gets data from DynamoDB and **caches it**.
 - Next read? 🚀 → DAX serves data **from memory**.
 - Writes/Updates? ✍️ → DAX updates both **cache and DynamoDB** (write-through).
-



Example Use Case



E-Commerce Website

Product catalog reads are frequent but updates are rare.



Use DAX to serve read requests like:

```
SELECT * FROM products WHERE category = 'laptops'
```

This avoids repeated hits to DynamoDB and speeds up the app significantly.



Performance Benefits

Without DAX	With DAX
~5–10 ms latency	 ~1 ms latency
DynamoDB API handles all reads	Cache serves repeated reads
More expensive for frequent queries	Saves cost on read-heavy workloads

How to Set Up DAX (Steps)

1.  Go to **DynamoDB > DAX**
 2.  Click **Create Cluster**
 - Set cluster name, node type, replication factor (1–10 nodes)
 3.  Set VPC, subnets, security groups
 4.  IAM Role with permission: [AmazonDynamoDBFullAccess](#) + [AmazonDAXFullAccess](#)
 5.  Update your SDK to point to DAX:
 - Python Boto3, Java, Node.js, etc.
 - SDKs use the DAX endpoint instead of DynamoDB
-

When Not to Use DAX

Scenario	Why Not
----------	---------

 Write-heavy apps	Cache won't help write speed
 Infrequent reads	Cold cache → no benefit
 Strict consistency required	DAX uses eventual consistency
 Custom caching logic needed	Use Redis/Memcached instead

Security & Monitoring

-  IAM for access control
 -  VPC for network isolation
 -  CloudWatch for DAX metrics (cache hit rate, latency)
 -  Supports encryption at rest with KMS
-

Pricing Notes

- Charged per **DAX node-hour**
- Additional charges for:
 - Data transfer
 - Multi-AZ replication

- Use **AWS Pricing Calculator** to estimate cost
-

✅ Summary Table

Feature	DAX
Type	In-memory cache for DynamoDB
Speed	Microsecond latency
Consistency	Eventual only
Ideal Use Case	Read-heavy, low-update workloads
Setup Time	~5 mins
Code Changes Needed	Minimal (SDK config only)

🎓 Bonus: Student Exercise Idea

Lab: Create a DynamoDB table + DAX cluster

Task: Query product info with and without DAX

Compare: Response times via logs or metrics

Outcome: Understand caching impact on performance
